

## 542. 01 Matrix Medium

Acceptance: 53.1% | [LeetCode](#)

Given an  $m \times n$  binary matrix `mat`, return *the distance of the nearest 0 for each cell*.

The distance between two cells sharing a common edge is 1.

**Example 1:**

```
**Input:** mat = [[0,0,0],[0,1,0],[0,0,0]]  
**Output:** [[0,0,0],[0,1,0],[0,0,0]]
```

**Example 2:**

```
**Input:** mat = [[0,0,0],[0,1,0],[1,1,1]]  
**Output:** [[0,0,0],[0,1,0],[1,2,1]]
```

**Constraints:**

- `m == mat.length`
- `n == mat[i].length`
- `1 <= m, n <= 104`
- `1 <= m * n <= 104`
- `mat[i][j]` is either 0 or 1.
- There is at least one 0 in `mat`.

**Note:** This question is the same as 1765: <https://leetcode.com/problems/map-of-highest-peak/>

Claude Opus Solution

## 01 Matrix — Solution Guide

### 1. Intuition

The key insight is recognizing this as a **shortest-path-from-multiple-sources** problem:

- We want the distance from **every cell** to its **nearest 0-cell**.
- Instead of asking "for each 1, where is the closest 0?" (which would mean many separate searches), we **flip the question**: start from **all 0-cells simultaneously** and let the distance "spread outward" layer by layer.

This is a classic **Multi-Source BFS**. Because BFS explores cells in increasing distance order, the first time we reach any cell, we've found its shortest distance. By seeding the queue with *all* zeros at once, the wavefronts from every zero expand together, and each 1-cell is naturally claimed by the closest zero.

**Pattern recognition:** "Distance to nearest X in a grid where movement is unit-cost" → **Multi-Source BFS**.

The reference solution uses a clever **DP two-pass** alternative (explained at the end), but BFS is the most intuitive standard approach.

### 2. Approach (Pseudocode)

- Initialize a `dist` matrix:
  - `dist[i][j] = 0` if `mat[i][j] == 0` (these are the BFS sources)
  - `dist[i][j] = unvisited` (use `-1` or `None`) otherwise
- Push all 0-cells into a queue.
- BFS:
  - while queue not empty:
    - `(r, c) = queue.popleft()`
    - for each of 4 neighbors `(nr, nc)`:
      - if neighbor is in bounds AND not visited yet:
        - `dist[nr][nc] = dist[r][c] + 1` # one step further
        - push `(nr, nc)` into queue
- Return `dist`.

### 3. Python Solution

```
from collections import deque  
from typing import List
```

```

class Solution:
    def updateMatrix(self, mat: List[List[int]]) -> List[List[int]]:
        m, n = len(mat), len(mat[0])

        # dist[i][j] will hold the answer (distance to nearest 0).
        # We use -1 as a "not visited yet" marker for 1-cells.
        dist = [[-1] * n for _ in range(m)]

        # Queue holds cells whose distance is already finalized.
        # We seed it with ALL 0-cells (multi-source BFS).
        queue = deque()
        for i in range(m):
            for j in range(n):
                if mat[i][j] == 0:
                    dist[i][j] = 0          # distance from a 0 to itself is 0
                    queue.append((i, j))    # source of the BFS wavefront

        # 4-directional movement vectors (up, down, left, right)
        directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

        # Standard BFS. Because all sources start at distance 0 and we
        # expand level by level, the first time a cell is reached is via
        # its shortest path to ANY 0.
        while queue:
            r, c = queue.popleft()
            for dr, dc in directions:
                nr, nc = r + dr, c + dc
                # Process neighbor only if in bounds and not yet visited.
                # dist == -1 means we haven't assigned it a distance yet,
                # which guarantees we never overwrite a shorter distance.
                if 0 <= nr < m and 0 <= nc < n and dist[nr][nc] == -1:
                    dist[nr][nc] = dist[r][c] + 1  # one step from current cell
                    queue.append((nr, nc))          # enqueue for further expansion

        return dist

```

## 4. Complexity Analysis

| Metric | Complexity | Explanation                                                                                                         |
|--------|------------|---------------------------------------------------------------------------------------------------------------------|
| Time   | $O(m * n)$ | Each cell is enqueued and dequeued exactly once. For each cell we inspect its 4 neighbors → constant work per cell. |
| Space  | $O(m * n)$ | The <code>dist</code> matrix plus the BFS queue, which in the worst case (all zeros) holds all cells.               |

Note: The `dist` output is required anyway, so the extra auxiliary space beyond the output is the queue — also  $O(m*n)$  worst case.

## 5. Alternative Approaches

### A. Dynamic Programming — Two-Pass (the reference solution)

The "distance to nearest 0" for a cell can only come from one of its 4 neighbors being closer. A single pass can only "see" two directions (e.g., top & left), so we do **two passes**:

- **Pass 1 (top-left → bottom-right)**: absorb best distance from *up* and *left* neighbors.
- **Pass 2 (bottom-right → top-left)**: absorb best distance from *down* and *right* neighbors.

After both passes, every cell has considered all 4 directions.

```

def updateMatrix(self, mat):
    m, n = len(mat), len(mat[0])
    INF = float('inf')
    dist = [[0 if mat[i][j] == 0 else INF for j in range(n)] for i in range(m)]

    # Forward pass: check up and left
    for i in range(m):
        for j in range(n):
            if i > 0: dist[i][j] = min(dist[i][j], dist[i-1][j] + 1)
            if j > 0: dist[i][j] = min(dist[i][j], dist[i][j-1] + 1)

```

```
# Backward pass: check down and right
for i in range(m-1, -1, -1):
    for j in range(n-1, -1, -1):
        if i < m-1: dist[i][j] = min(dist[i][j], dist[i+1][j] + 1)
        if j < n-1: dist[i][j] = min(dist[i][j], dist[i][j+1] + 1)

return dist
```

- **Time:**  $O(m * n)$  — two linear passes.
- **Space:**  $O(1)$  extra (ignoring the output matrix). **This is the main advantage over BFS** — no queue needed.

## B. Why not single-source BFS per cell?

Running BFS from each 1-cell separately would be  $O((m*n)^2)$  — far too slow. Multi-source BFS collapses this into a single  $O(m*n)$  sweep.

## Summary

- **BFS approach:** most intuitive, generalizes well to weighted/Dijkstra variants. Use when clarity matters.
- **DP two-pass:** elegant,  $O(1)$  extra space, works here *only because* movement is unit-cost in a grid with axis-aligned moves. Prefer when memory is tight.

Both are optimal at  $O(m*n)$  time.

## Codex Solution

### 1. Intuition

For every cell:

- If `mat[r][c] == 0`, its distance is 0.
- If `mat[r][c] == 1`, its nearest 0 must be reached through one of its 4 neighbors.

So the recurrence is:

```
dist[r][c] = 1 + min(
    dist[r - 1][c],
    dist[r + 1][c],
    dist[r][c - 1],
    dist[r][c + 1]
)
```

The challenge is that this depends on all directions, which can create cycles.

Key insight:

- We can solve this using **two dynamic programming passes**.
- First pass: top-left to bottom-right
- Handles information coming from **top** and **left**.
- Second pass: bottom-right to top-left
- Handles information coming from **bottom** and **right**.

After both passes, every cell has considered all 4 directions.

## 2. Approach

### Pseudocode

```
initialize dist matrix with infinity

for each cell:
    if mat[r][c] == 0:
        dist[r][c] = 0

first pass: top-left to bottom-right
for r from 0 to m - 1:
    for c from 0 to n - 1:
        if r > 0:
            dist[r][c] = min(dist[r][c], dist[r - 1][c] + 1)
```

```

        if c > 0:
            dist[r][c] = min(dist[r][c], dist[r][c - 1] + 1)

second pass: bottom-right to top-left
for r from m - 1 down to 0:
    for c from n - 1 down to 0:
        if r < m - 1:
            dist[r][c] = min(dist[r][c], dist[r + 1][c] + 1)
        if c < n - 1:
            dist[r][c] = min(dist[r][c], dist[r][c + 1] + 1)

return dist

```

### 3. Python Solution

```

from typing import List

class Solution:
    def updateMatrix(self, mat: List[List[int]]) -> List[List[int]]:
        m = len(mat)
        n = len(mat[0])

        # The maximum possible distance in the grid is at most m + n - 2.
        # So any large number greater than that works as infinity.
        INF = m + n

        # dist[r][c] will store the distance from cell (r, c)
        # to the nearest 0.
        dist = [[INF for _ in range(n)] for _ in range(m)]

        # Initialize all 0-cells with distance 0.
        # All 1-cells remain INF for now.
        for r in range(m):
            for c in range(n):
                if mat[r][c] == 0:
                    dist[r][c] = 0

        # First pass: top-left to bottom-right.
        #
        # At each cell, we only know reliable information from:
        # - the top neighbor
        # - the left neighbor
        #
        # This pass propagates distances from zeros that can be reached
        # through top/left directions.
        for r in range(m):
            for c in range(n):
                # Check top neighbor.
                if r > 0:
                    dist[r][c] = min(dist[r][c], dist[r - 1][c] + 1)

                # Check left neighbor.
                if c > 0:
                    dist[r][c] = min(dist[r][c], dist[r][c - 1] + 1)

        # Second pass: bottom-right to top-left.
        #
        # Now we consider the remaining two directions:
        # - the bottom neighbor
        # - the right neighbor
        #
        # Combining both passes means every cell has considered all 4 neighbors.
        for r in range(m - 1, -1, -1):
            for c in range(n - 1, -1, -1):
                # Check bottom neighbor.
                if r < m - 1:
                    dist[r][c] = min(dist[r][c], dist[r + 1][c] + 1)

                # Check right neighbor.
                if c < n - 1:

```

```
dist[r][c] = min(dist[r][c], dist[r][c + 1] + 1)

return dist
```

## 4. Complexity Analysis

Let:

```
m = number of rows
n = number of columns
```

### Time Complexity

$O(m * n)$

We scan the matrix three times:

1. Initialization
2. First DP pass
3. Second DP pass

Each scan costs  $O(m * n)$ .

So total time is:

$O(m * n)$

### Space Complexity

$O(m * n)$

We create a separate `dist` matrix of size  $m \times n$ .

If modifying the input matrix were allowed, we could store distances directly inside `mat`, reducing extra space to  $O(1)$  beyond the output.

## 5. Alternative Approaches

### Multi-source BFS

Another common solution is **multi-source BFS**.

Idea:

- Push all `0` cells into a queue initially.
- Start BFS from all zeros at once.
- The first time we reach a `1` cell, we have found its nearest distance to a zero.

This works because BFS explores cells in increasing distance order.

Complexity is also:

```
Time: O(m * n)
Space: O(m * n)
```

### Comparison

| Approach         | Time    | Space   | Notes                                                        |
|------------------|---------|---------|--------------------------------------------------------------|
| Two-pass DP      | $O(mn)$ | $O(mn)$ | Very clean for this grid-distance problem                    |
| Multi-source BFS | $O(mn)$ | $O(mn)$ | More general, works well for shortest path from many sources |